Meta Reinforcement Learning for Multi-Task Offloading in Vehicular Edge Computing

Penglin Dai, *Member, IEEE*, Yaorong Huang, Kaiwen Hu, Xiao Wu, *Member, IEEE*, Huanlai Xing, *Member, IEEE*, and Zhaofei Yu, *Member, IEEE*

***Abstract*—Mobile edge computing has been a promising solution to enable real-time service in vehicular networks. However, due to high dynamics of mobile environment and heterogeneous features of vehicular services, traditional expert-based or learning-based strategies has to update handcrafted parameters or retrain learning model, which leads to intolerant overhead. Therefore, this paper investigates the problem of multi-task offloading (MTO), where there exist multiple offloading scenarios with varying parameters, such as task topology, resource requirement and transmission/computation capability. The objective is to design a unified solution to minimize task execution time under different MTO scenarios. Accordingly, we develop a Seq2seq-based Meta Reinforcement Learning algorithm for MTO (SMRL-MTO). Specifically, a bidirectional gated recurrent units integrated with attention mechanism is designed to determine offloading action by encoding sequential offloading actions and showing different preferences to different parts of input sequence. Particularly, a meta reinforcement learning framework is designed based on model-agnostic meta learning, which trains a meta policy offline and fast adapts to new MTO scenario within a few training steps. Finally, we conduct performance evaluation based on task generator DAGGEN and realistic vehicular traces, which shows that the SMRL-MTO reduces task execution time by 11.36% on average compared with greedy algorithm.**

***Index Terms*—Meta reinforcement learning, multi-task offloading, vehicular edge computing, multi-task dependency, Seq2seq model.**

I. Introduction

MOBILE edge computing (MEC) has been a promising solution to enable real-time vehicular services by providing computation, communication and caching resources at network edge. In these vehicular services, such as augmented reality service and vehicular navigation application, mobile vehicles usually generate computation-intensive tasks, where each task is modeled as directed acyclic graph (DAG). Due to the limited local computation capability, vehicles have to partially or fully offload tasks to nearby MEC server via wireless communication for reducing computation time. However, mobile vehicles still suffer from serious delay in MEC-based architecture due to the following observations. First, different vehicular tasks have different requirements, which cannot be satisfied by a single dedicated designed strategy. Second, the capabilities of MEC servers are heterogeneous and vary with different scenarios, which easily causes unbalanced workload distribution. Therefore, it is still non-trivial to design an effective offloading mechanism adaptive to diverse service demands of vehicular applications simultaneously.

In the last decades, great efforts have been paid on task offloading in vehicular networks. Some researchers investigated on different types of service architectures. [1], [2] studied the cloud-based vehicular networks, where the computation-intensive tasks generated by vehicles can be wirelessly offloaded to remote cloud server for reducing computation time. However, frequent information exchange between terminal users and cloud server brings great backhaul traffic and causes excessive transmission delay. To reduce transmission overhead, [3], [4] proposed the MEC-based vehicular networks, where vehicular tasks can be directly offloaded to the nearby MEC servers deployed at wireless infrastructures, such as Base Stations (BSs) and Roadside Units (RSUs). To further improve system performance, [5], [6] studied the MEC/Cloud hybrid service architecture, where mobile vehicles, MEC and cloud servers, are cooperated to satisfy dynamic resource requirements of vehicular tasks. Further, to fully exploit the heterogeneous resources, some researchers developed different types of task offloading mechanisms in vehicular networks. [6], [7] developed several heuristic or approximation algorithms based on the assumption that the knowledge of tasks and MEC servers is known in advance. [8], [9] applied reinforcement learning-based algorithms to explore optimal solution without knowing environmental information. However, the features of both vehicular task and environment vary a lot with different service scenarios in vehicular networks. In particular, some vehicles may not fully reveal the features of tasks as well as other critical information during data offloading due to the privacy protection. Further, these works only considered a simple task

Manuscript received 29 October 2022; revised 3 February 2023; accepted 17 February 2023. Date of publication 23 February 2023; date of current version 5 February 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62172342, in part by the Natural Science Foundation of Hebei Province, China under Grant F2022105003, in part by the Key Project of Science and Technology Research Program of Chongqing Education Commission of China under Grant KJZD-K202202901, in part by the Fundamental Research Funds for the Central Universities under Grant 2682022JX007, in part by the China Postdoctoral Science Foundation funded project under Grants 2019M660245 and 2020T130547. Recommended for acceptance by Dr. D. Pompili. *(Corresponding author: Penglin Dai.)*

Penglin Dai, Yaorong Huang, Kaiwen Hu, Xiao Wu, and Huanlai Xing are with the School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu 611756, China, and with the Engineering Research Center of Sustainable Urban Intelligent Transportation, Ministry of Education, Chengdu 611756, China, and also with the Tangshan Institute of Southwest Jiaotong University, Tangshan 063000, China (e-mail: penglindai@swjtu.edu.cn; huangyaorong@my.swjtu.edu.cn; kaiwenhu_swjtu@163.com; wuxiaohk@gmail.com; hxx@home.swjtu.edu.cn).

Zhaofei Yu is with the Institute for Artificial Intelligence, Peking University, Beijing 100871, China (e-mail: yzf714@126.com).

Digital Object Identifier 10.1109/TMC.2023.3247579

1536-1233 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

offloading scenario, where the resource competition between multiple vehicles and the workload balance between multiple wireless channels as well as processors are neglected.

Based on the above motivation, we investigate multi-task offloading (MTO) problem in vehicular edge computing (VEC), where there exist multiple task offloading scenarios characterized by varying features of both vehicular tasks and MEC servers. Specifically, the tasks generated from mobile vehicles are computation-intensive and abstracted as a directed acyclic graph (DAG) and divided into multiple dependent subtasks with different resource requirements, where each subtask can be processed only when the computation server receives the data of subtask itself and the output data from its parent subtasks. In this paper, the subtask is the basic offloading unit and can be either processed by local vehicle or wirelessly offloaded to nearby MEC server. Compared with local vehicles, the MEC server has more powerful computation capabilities and owns multiple wireless subchannels and processors, which undertakes overloaded tasks from the vehicles for effectively reducing task execution time. However, it is still challenging to design a unified offloading strategy fast adaptive to the MTO scenarios with varying parameter settings. First, the task profiles, including task topology and resource requirement, are always location-aware and time varying, which is difficult to infer task execution time. Second, the resource competition between multiple vehicles may cause unbalanced workload distribution among heterogeneous channels and processors and result in excessive transmission or computation time.

To resolve the above issues, we develop a Seq2seq-based Meta Reinforcement Learning algorithm for MTO (SMRL-MTO), where the seq2seq model and the meta reinforcement learning (MRL) are used for predicting task execution time and fast adapting to new offloading scenario, respectively. The contributions of this work are outlined as follows.

- We formulate a problem of multi-task offloading (MTO) in the VEC system, where vehicular tasks are modeled as DAG-based graphs and MEC servers are characterized by heterogeneous transmission and computation capabilities. Particularly, the features of both vehicular tasks and MEC servers are location-aware and time-varying. The task offloading model consists of transmission and computation, where the detail procedure are managed by multiple wireless subchannels and computation processors, respectively. The objective of the MTO is to derive a unified solution to minimize task execution time under different scenarios with varying parameter settings simultaneously.
- We develop the Seq2seq-based Meta Reinforcement Learning for MTO (SMRL-MTO), which can efficiently find an offloading policy adaptive to a new environment. Specifically, we design a Seq2seq model by combining a Bi-GRU with attention mechanism, which explores the relationships between subtasks by encoding sequential offloading actions and makes offloading action by showing different preferences to different parts of input sequence. Further, a meta-learning framework consists of local gradient update and meta training are designed, where meta training obtains the meta policy by offline training and local gradient update achieves a dedicated offloading policy by running meta offloading policy with a few epochs.
- We build a comprehensive simulation model by integrating realistic vehicular traces, open-source DAG generator DAGGEN and the VEC model. The simulation results demonstrate the effectiveness and the adaptiveness of the proposed algorithm compared to three competitive algorithms in a wide range of service scenarios.

The remainder of this article is organized as follows. In Section III, the system model is presented. In Section IV, we formulate the MTO problem. In Section V, we present the algorithm design. Section VI gives a comprehensive performance evaluation. Finally, Section VII concludes this article.

II. Related Work

Recently, many efforts have been paid on investigating the service architecture of task offloading in vehicular networks, which are generally classified into three categories: cloud-based, MEC-based and hybrid architectures. In the cloud-based architecture, researchers considered how to offload the task from the mobile vehicles to the cloud server to reduce the computation load of local vehicular terminal. [10] proposed a dynamic programming with randomization algorithm that iteratively improves the offloading decision vector to offload as many tasks as possible to the cloud server. [11] designed a dynamic method to offload specific vehicle application components to the cloud through dynamic decision-making under variable network conditions for reducing the response time. However, as the vehicular terminal is far away from the cloud server, the task suffers from excessive transmission delay during offloading procedure. In order to overcome this issue, some researchers proposed the MEC-based service architecture, where the MEC server is deployed at network edge for processing real-time vehicular tasks without accessing to the cloud. [12] designed an MEC-MEC cooperation architecture and designed a non-cooperative game-theoretic strategy to realize the MEC grouping for workload balance between MEC servers. [13] designed an MEC-cloud cooperation architecture and achieved the load balance among MEC and cloud servers by making different offloading decisions for tasks with different resource demand and delay sensitivity. In addition, some researchers considered to utilize the computation resource of vehicle. [14] proposed a V2V hybrid multi-hop edge computing offloading algorithm to offload the tasks to the vehicle out of the coverage of RSU for reducing overloaded issue. [15] studied a mobility-aware computation offloading algorithm to reduce the total cost by comparing the average cost of running task in local vehicle and MEC server, respectively. [16] studied a scenario where the tasks generated by sensor device are offloaded to one of the local vehicle, MEC server or cloud based on a deep learning model, which effectively reduces energy consumption and delay.

Further, researchers investigated on the mechanisms of computation offloading and resource allocation for improving the offloading performance. Many researchers focused on designing computation offloading strategies with respect to different objectives, such as execution delay and energy consumption.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

[17] formulated a cooperative task offloading problem in MEC-based architecture as a mixed-integer optimization problem, and proposed an iterative algorithm based on the Lagrangian dual decomposition for minimizing the latency. [18] proposed an optimization algorithm based on deep reinforcement learning for minimizing the energy consumption while satisfying the delay constraint of users based on a constructed three-layer offloading architecture. In addition, some researchers focused on investigating resource allocation mechanisms, which also has a crucial impact on task offloading performance. With respect to communication resource, [19] analysed the queuing system for the V2V link and derived the packet loss probability and average packet sojourn time under fixed power allocation, which achieves the optimal power allocation for each possible pair. With respect to computation resource, [20] formed a container-based multimedia collaboration system, and designed a container layout algorithm to minimize the overall system cost. Further, some researchers investigated the joint optimization model by considering both computation offloading and resource allocation. [21] formulated a joint offloading optimization model in virtual reality application, and proposed a centralized algorithm to decide the computation offloading action and resource allocation ratio to minimize the completion delay. [22] proposed a distributed joint optimization algorithm to provide the optimal offloading action and resource allocation strategies, which obtains the minimum cost for all mobile devices.

In addition, great efforts have been paid on designing a variety of task offloading mechanisms dedicated for different environmental settings. Some researchers developed heuristic algorithms by synthesizing multiple tasks and environmental features. [23] formulated sequential task offloading as multi-objective optimization and designed a heuristic algorithm based on the evaluation of offloading order of MEC server with synthesizing computation capability and transmission link quality. [24] the problem of dependency-aware task offloading in ultra-dense edge networks and designed a priority-based algorithm, where the priority of each subtask is evaluated by the weighted sum of computation overhead and local energy consumption. [25] developed an adaptive computation offloading method in 5G-envisioned vehicular networks for optimizing task offloading delay and resource utilization of the edge system, which leverages multi-objective evolutionary algorithm based on decomposition (MOEA/D) to generate the pareto-optimal solutions. Further, some researchers formulated task offloading problem as a mathematical model and designed optimization algorithm based on convex or nonconvex theory. [26] formulated the problem of cooperative computation offloading based on queuing theory, and proposed a probabilistic computation offloading algorithm based on Alternating Direction Method of Multipliers (ADMM), which converges to the optimal solution in an iterative way. However, these heuristic or optimization-based algorithms rely heavily on expert knowledge or accurate mathematical models, where expert knowledge and model parameters have to be periodically updated due to the dynamic vehicular environment. To overcome this issue, reinforcement learning-based [18], [27] algorithms are developed to learn effective offloading solution by trial and error without accurate information of the environment. [28] proposed an online learning framework to guide task offloading in a small cell network by leveraging multi-armed bandit technique to update task weights according to the feedback in uncertain and stochastic environment, which achieves a close-to-optimal reward with low violations, and outperforms many state-of-the-art algorithms. [29] proposed a knowledge driven service offloading decision framework, which explores deep reinforcement learning to obtain the optimal solution by scrupling future data dependence of following tasks based on the knowledge of current task. However, these algorithms have weak adaptability for unexpected environmental fluctuation, such as task topology, task number, computation and communication resources, which requires full retraining to learn the policy for a new environment with intolerant time consumption. To accommodate to different task scenarios, several fast adaptation frameworks are developed to obtain the specific policy to a new task within relatively small overhead. Particularly, [30], [31] investigated online task scheduling in vehicular networks with dynamic topology and unstable connection, and proposed an imitation learning algorithm with near-optimal performance from the initial stage, where the optimal scheduling policy is obtained by following the expert's demonstration with a few samples. [32] developed a gradient-based meta reinforcement learning that is able to adapt the nature of the return, such as the rate of reward discount and the optimal reward form, whilst interacting and learning from the dynamic environment.

Distinguished from previous works, this paper adopts the framework of model-agnostic meta-learning (MAML) [33], where the basic idea is to train meta parameter instead of exploring optimal reward form such that a small number of gradient updates will lead to fast learning on a new task. Compared with other fundamental meta-RL, such as RL^2 [34], MAESN [35] and PEARL [36], the MAML is a simple general framework and easy to be integrated with our task encoding model, which is suitable for our concerned application. Particularly, the closest work to our paper is [37], which also applied the MAML framework to DAG-based task offloading in the scenario of a single user with the prior knowledge of task topology. Particularly, a sequence-to-sequence (seq2seq) neural network is exploited to model the offloading policy, where the LSTM-based encoder is designed for capturing task topology and the PPO-clip based policy objective is used for model training. Compared with [37], this paper focused on a comprehensive multi-task offloading scenario, where a new MDP model is designed to characterize the property of multiple-vehicle resource competition and the model of multiple wireless channels and computation servers. Particularly, a new encoder based on BiGRU is designed in seq2seq network for task embedding within implicit relation between subtasks. Further, the policy objective is designed by synthesizing the PPO-clip and KL divergence for stabilizing the training process. Compared with existing learning-based scheduling algorithms, our work aims at achieving better minimization of task execution time within fewer gradient steps by utilizing seq2seq model for task structure exploration and the MRL for meta-policy training. As far as we know, it is the first

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

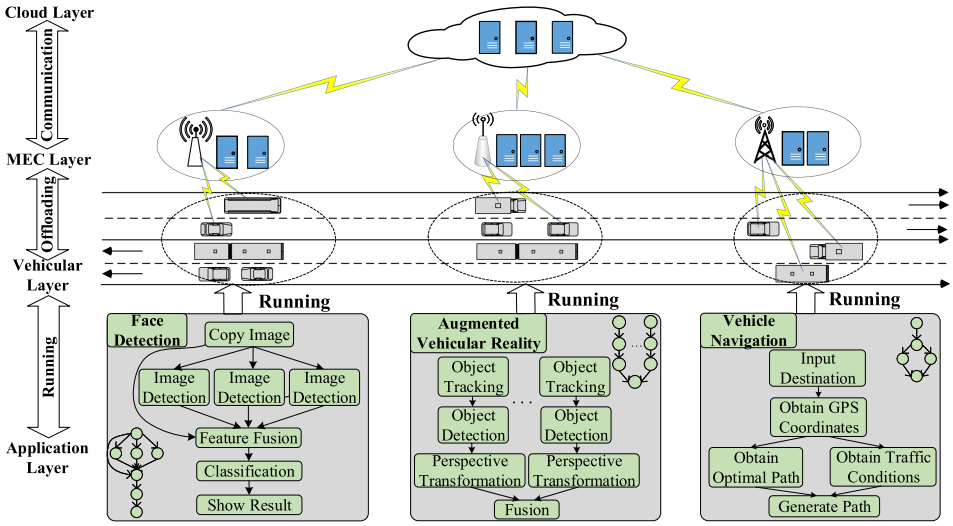


Fig. 1. The system architecture of multi-task offloading in MEC-based vehicular network.

work to investigate the MTO problem in MEC-based vehicular networks by considering dynamic features of both vehicular tasks and MEC servers.

III. System Model

In this section, we present system architecture of multi-task offloading in MEC-based vehicular network in the perspective of application layer, vehicular layer and MEC layer, respectively, as shown in Fig. 1.

In application layer, there exist various types of vehicular tasks, such as face recognition, augmented vehicular reality and vehicular navigation, where each vehicular task can be divided into multiple dependent functional modules. For instance, augmented vehicular reality can be divided into multiple dependent functional modules, such as object tracking, object detection, perspective transformation and fusion. Accordingly, each vehicular task is abstracted as a directed acyclic graph (DAG), where the vertex represents the subtask and the directed edge represents the dependency between two subtasks. In this paper, the successor and predecessor subtasks are called parent and child, respectively. The child subtask can start to run only when receiving the computation results of all its parent subtasks as well as the information related to subtask itself. The task features are characterized by graph topology, required computation resources and the size of dependent data transmitted between subtasks, which varies with the types of vehicular tasks.

In vehicular layer, each vehicle is assumed to run one type of vehicular task and can process part of task within its local processor. Due to its limited computation capability, the vehicle has to choose to offload the overloaded subtasks to the nearby MEC server via wireless communication. Particularly, the vehicle has two queues: the local computation queue and the local transmission queue, which are used for storing the pending subtasks to be processed by local processor and transmitted to MEC servers, respectively. The computation and transmission operations can

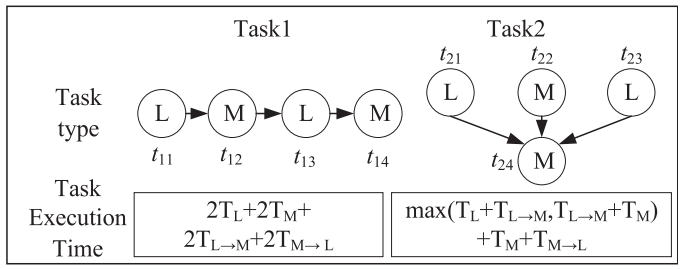


Fig. 2. The diagram of the MTO.

be performed simultaneously. Before task offloading, each vehicle will encapsulate the relevant information into the beacon message, such as required computation resources, and the size of subtask itself, and periodically send to the MEC server.

In MEC layer, the MEC server is deployed at wireless infrastructure and assumed to own multiple wireless subchannels and processors, which can provide communication and computation services for passing vehicles. Further, the MEC server plays the role of the scheduler and periodically makes the offloading decision of each pending task submitted by vehicles within its coverage. Specifically, the MEC server will receive submitted tasks by overhearing the beacon message broadcast by vehicles. Then, at the beginning of each time slot, the MEC server sorts all the pending subtasks based on a designed mechanism and makes the offloading decision of each subtask in a sequential order.

We give a simple example to illustrate the characteristics of MTO. As shown in Fig. 2, we have two vehicular tasks, which both have four subtasks but with different typologies. For simplification, the subasks are assumed to be homogeneous, which require the same computation resources and generate the same size of computation results. The subtasks of task 1 are executed in a sequential order and the notation L (or M)

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF NOTATIONS

Notation	Description
V	The set of MEC servers
P	The set of subtasks
M	The set of processors owned by MEC server
R	The set of subchannels owned by MEC server
$t_{v,p}$	the p_{th} subtask owned by vehicle v.
$c_{v,p}$	The required computation resource of $t_{v,p}$.
$d^I_{v,p}$	The input data size of $t_{v,p}$.
$d^L_{v,p}$	The size of substak itself of $t_{v,p}$.
$d^O_{v,p}$	The output data size of $t_{v,p}$.
$x^r_{v,p}$	=1, which indicates that the subtask $t_{v,p}$ is offloaded via the r_{th} subchannel.
$y^m_{v,p}$	=1, which indicates that the subtask $t_{v,p}$ is offloaded to the m_{th} processors.
$P(t_{v,p})$	The set of the parent subtasks of $t_{v,p}$.
$P^L(t_{v,p})$	The parent subtasks of $t_{v,p}$ offloaded to local processor .
$P^M(tv,p)$	The parent subtasks of $t_{v,p}$ offloaded to the MEC server.
$TS(d^O_{v,p})$	The start time of transmitting output data $d^O_{v,p}$ of $t_{v,p}$.
$T(d^O_{v,p})$	The time duration of transmitting output data $d^O_{v,p}$ of $t_{v,p}$.
$TE(t_{v,p})$	The time of completely receiving the input data of subtask $t_{v,p}$.
$CS(t_{v,p})$	The start time of computing the subtask $t_{v,p}$.
$C(t_{v,p})$	The time duration of computing the subtask $t_{v,p}$.
$CE(t_{v,p})$	The time of completely computing the subtask $t_{v,p}$.

indicates that the subtask is processed by local vehicle (or offloaded to MEC server). For example, t_{11} is processed by the local vehicle and the computation time is denoted by T_L. t_{12} is offloaded to MEC server, which consumes upload time $T_{L\to M}$ and computation time T_M. Further, the computation result of t_{12} has to be transmitted to t_{13}, which consumes download time $T_{M\to L}$. Accordingly, the execution time of task 1 is $2(T_L + T_M + T_{L\to M} + T_{M\to L})$. Task 2 has three parallel subtasks t_{21}, t_{22} and t_{23}. t_{24} can start to run when receiving the computation results of all its parents. Therefore, the execution time of t_2 is $MAX(T_L + T_{L\to M}, T_M + T_{L\to M}) + T_M + T_{M\to L}$. Based on the above observation, the topology of vehicular task has great effect on task execution time. Further, both the subtasks and MEC server can be heterogeneous, which varies with different vehicular applications. Therefore, a fast adaptive offloading mechanism is urgently needed for dynamic vehicular environment.

IV. PROBLEM FORMULATION

A. Preliminary

Each task offloading scenario is denoted by $\mathcal{T}_i$ and characterized by the features of both vehicular tasks and MEC servers. In each task offloading scenario $\mathcal{T}_i$, the set of vehicles is denoted by V. Each vehicle $v \in V$ is assumed to run one task modeled as a DAG, which consists of a set of subtasks and a set of directed edges represent for the dependencies between two substasks. The p_{th} subtask on vehicle v is denoted by $t_{v,p}$, which is characterized by a three-tuple $(d^L_{v,p}, c_{v,p}, d^O_{v,p})$. Specifically, $d^L_{v,p}$ is the size of subtask itself (i.e., the program code and related parameters), $c_{v,p}$ is the required computation resource and $d^O_{v,p}$ is the size of its output data. Further, the input data of a subtask $t_{v,p}$, denoted by $d^I_{v,p}$, is defined as the summation of the subtask itself and the output data of its parents, i.e.,

$$d^I_{v,p} = d^L_{v,p} + \sum_{t_{v,k}\in P(t_{v,p})} d^O_{v,k}, \forall v \in V, \forall p \in P, \quad (1)$$

where $P(t_{v,p})$ denotes the set of its parent subtasks. It is assumed that the subtasks share the output data of the same parents. On the other hand, the MEC server is characterized by its transmission and computation capability. The MEC server has R uplink subchannels, where wireless bandwidth of the r_{th} subchannel is denoted by w_r. The MEC server owns M processors and the computation frequency of m_{th} processor is f_m. The primary notations used in this paper are summarized in Table 1.

In addition, the offloading decision of subtask $t_{v,p}$ is represented by a set of binary integer variables $x^r_{v,p}$ and $y^m_{v,p}$, where $x^r_{v,p}$ and $y^m_{v,p}$ indicate whether the r_{th} subchannel and the m_{th} processor are selected for task offloading or not, respectively. Particularly, $x^0_{v,p} = 1$ or $y^0_{v,p} = 1$ indicates that the subtask is processed by the local vehicle. The first-in-first-service (FIFS) queuing model is adopted to simulate the process procedure of computation processor and transmission channel. As the subtask is the minimum offloading unit, only one subchannel and one processor can be selected for each subtask $t_{v,p}$. Therefore, the following constraints must be satisfied.

$$\begin{aligned}
&\sum_{r\in R\cup\{0\}} x^r_{v,p} = 1, \quad \forall v \in V, \forall p \in P,\\
&\sum_{m\in M\cup\{0\}} y^m_{v,p} = 1, \quad \forall v \in V, \forall p \in P,\\
&x^0_{v,p} = y^0_{v,p}, \quad \forall v \in V, \forall p \in P,\\
&x^r_{v,p}, y^m_{v,p} \in \{0,1\}, \quad \forall r \in R\cup\{0\}, \forall m \in M\cup\{0\} \quad (2)
\end{aligned}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

B. Task Offloading Model

In this section, we formulate the task offloading model, which consists of task transmission and computation model.

First, the input data of subtask $t_{v,p}$ has to be transmitted to the offloaded server. To facilitate further discussion, we divide $P(t_{v,p})$ into two sets: the parents offloaded to MEC server and local processor, denoted by $P^M(t_{v,p})$ and $P^L(t_{v,p})$, respectively. Let $TE(t_{v,p})$ denote the time of completely receiving the input data of subtask $t_{v,p}$, which is analyzed in the following two cases.

- When the subtask $t_{v,p}$ is offloaded to local vehicle, i.e., $x^0_{v,p} = 1$, only the output data of its parent subtasks in $P^M(t_{v,p})$, needs to be transmitted to the local vehicle via download channel. Once the subtask $t_{v,k}$ is completed, its output data $d^O_{v,k}$ is immediately put into the download queue and transmitted in a first-in-first-out (FIFO) rule. Let $CE(t_{v,k})$ denotes the time of completing the subtask $t_{v,k}$ and $WT^d(t_{v,k})$ denotes the waiting time in the download queue, then the start time of transmitting output data $d^O_{v,k}$ of $t_{v,k}$, denoted by $TS(d^O_{v,k})$, is computed by

$$TS(d^O_{v,k}) = CE(t_{v,k}) + WT^d(t_{v,k}), if\ x^0_{v,k} = 1, \quad t_{v,k} \in P^M(t_{v,p}). \tag{3}$$

  According to the Shannon's formula, the time of transmitting output data $d^O_{v,k}$ is computed as

$$T(d^O_{v,k}) = \frac{d^O_{v,k}}{w_d log_2\left(1 + \frac{q_m g_v}{\varpi}\right)}, if\ x^0_{v,k} = 1, \quad t_{v,k} \in P^M(t_{v,p}). \tag{4}$$

  where w_d is the download bandwidth, q_m is the transmission power, g_v is the channel gain between v and the MEC server, and ϖ is the noise power. On this basis, the time of receiving the input data of subtask $t_{v,p}$ denoted by $TE(t_{v,p})$, is computed as the maximum of the summation of $TS(d^O_{v,k})$ and $T(d^O_{v,k})$ among the subtasks $t_{v,k} \in P^M(t_{v,p})$, i.e.,

$$TE(t_{v,p}) = \max_{\forall t_{v,k} \in P^M(t_{v,p})} \{TS(d^O_{v,k}) + T(d^O_{v,k})\}, if\ x^0_{v,p} = 1. \tag{5}$$

- When the subtask $t_{v,p}$ is offloaded to the MEC server, i.e., $x^r_{v,p} = 1$, which indicates the r_{th} uplink subchannel is selected for data upload, then both the subtask itself and the output data of its parent subtasks in $P^L(t_{v,p})$ have to be offloaded to the MEC server. The local vehicle aggregates the output data of $P^L(t_{v,p})$ as well as the subtask itself together and then put them into the queue of uplink channel. In this case, we only care about the subtask $t^*_{v,p}$ with the maximum $CE(t^*_{v,p})$ among $P^L(t_{v,p})$, i.e.,

$$t^*_{v,p} = arg \max_{\forall t_{v,k} \in P^L(t_{v,p})} \{CE(t_{v,k})\}. \tag{6}$$

  Similarly, the start time of transmitting the output data of $t^*_{v,p}$ is formulated as follows

$$TS(t^*_{v,p}) = CE(t^*_{v,p}) + WT^r(t^*_{v,p}), \tag{7}$$

  where $WT^r(t^*_{v,p})$ is the waiting time of $t^*_{v,p}$ in the r_{th} uplink subchannel. Since the input data of $t_{v,p}$ is aggregated and transmitted together, $T(t^*_{v,p})$ is computed by the total size of input data of $t_{v,p}$ on the local vehicle divided by the upload rate of r_{th} uplink subchannel, which is computed as

$$T(t^*_{v,p}) = \frac{d^L_{v,p} + \sum_{\forall t_{v,k} \in P^L(t_{v,p})} d^O_{v,k}}{w_r log_2(1 + \frac{q_v g_v}{\varpi})}. \tag{8}$$

  Then $TE(t_{v,p})$ is computed as the summation of $TS(t^*_{v,p})$ and $T(t^*_{v,p})$, which is formulated as

$$TE(t_{v,p}) = TS(t^*_{v,p}) + T(t^*_{v,p}),\ if\ x^r_{v,p} = 1. \tag{9}$$

Second, after data transmission, the subtask $t_{v,p}$ is put into the computation queue of the local vehicle or the MEC server, which depends on its offloading decision. Let $WT^l(t_{v,p})$ (or $WT^m(t_{v,p})$) denote the waiting time of $t_{v,p}$ in the computation queue of local vehicle (or the MEC server), then the start time of computing the subtask $t_{v,p}$, denoted by $CS(t_{v,p})$, is computed as

$$CS(t_{v,p}) = \begin{cases} TE(t_{v,p}) + WT^l(t_{v,p}), & if\ y^0_{v,p} = 1, \\ TE(t_{v,p}) + WT^m(t_{v,p}), & if\ y^m_{v,p} = 1. \end{cases} \tag{10}$$

Then, the time duration of computing $t_{v,p}$ is computed by the required computation resource $c_{v,p}$ divided by the computation capacity of offloaded server, which is formulated as

$$C(t_{v,p}) = \begin{cases} \frac{c_{v,p}}{f_v}, & if\ y^0_{v,p} = 1, \\ \frac{c_{v,p}}{f_m}, & if\ y^m_{v,p} = 1. \end{cases} \tag{11}$$

Then, the time of completing subtask $t_{v,p}$ is computed as

$$CE(t_{v,p}) = CS(t_{v,p}) + C(t_{v,p}). \tag{12}$$

On this basis, the execution time of a vehicular task t_v is defined as the maximum of $CE(t_{v,p})$ among all the subtasks $t_{v,p} \in t_v$ minus the submission time $ST(t_{v,p})$, which is formulated as

$$ET(t_v) = \max_{\forall p \in P} \{CE(t_{v,p})\} - ST(t_{v,p}). \tag{13}$$

To evaluate system performance, we define average execution time (AET) as the objective, which is computed by the summation of the execution time of all vehicular tasks divided by the total task number, which is computed by

$$AET = \sum_{\forall v \in V} \frac{ET(t_v)}{||V||}. \tag{14}$$

C. Multi-Task Offloading Problem

Task offloading scenario is characterized by the parameters of both vehicular task and environment, including task topology, computation/computation resources, etc., where the specific values are assumed to be sampled from a given distribution within predefined range. Accordingly, task offloading scenarios differ from each other in the change of parameter value, which reflects

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

the dynamics of vehicular tasks or environment. For each $\mathcal{T}_i$, the set of parameters are given as follows: the DAG of vehicular task, the bandwidths of downlink and uplink subchannels, and the computation capabilities of local vehicle and MEC servers. Then, the optimization model is formulated as

$$\mathcal{T}_i: \min_{x^r_{v,p}, y^m_{v,p}} AET$$
$$s.t.\ Constraints\ in\ Eq.(2). \tag{15}$$

It is noted that the model in (15) is an integer linear programming (ILP) problem. Further, in the MTO problem, the parameters of vehicular tasks, local vehicles and MEC servers, vary with different offloading scenarios $\mathcal{T}_i$. Thus, these existing expert-based or learning-based algorithms suffer from overhigh overhead and cannot be fast adapted to dynamic application scenarios.

V. Algorithm Design

A. The Seq2Seq-Based MDP Model

We conceive the MTO problem as multiple MDPs and each scenario $\mathcal{T}_i$ is modeled as an MDP. The proposed SMRL-MTO algorithm aims to find an effective meta-policy for all MDPs as well as a fast adaptive offloading policy for a specific MDP. The basic elements of the MDP are designed as follows.

- System state: as observed in Section IV-B, the execution time of a subtask depends on task profile (such as required computation resources and data dependencies), the available communication and computation resources of MEC servers and the offloading decisions of previous subtasks. Accordingly, the system state at t step is formulated as

$$S = \{s_t | s_t = (e, a_{1:t}))\}, \quad t \in [1, N], \tag{16}$$

where e is the task embedding and $a_{1:t}$ is the offloading decision of the first t subtasks. To convert the DAG into a sequence, we index the subtasks according to its layer in the DAG and the scheduling rule is defined as follows. The subtask with lower layer is scheduled earlier and then have lower offloading index. For the subtasks with the same layer, the offloading indexes are randomly assigned. The subtask with lower offloading index can be scheduled in prior in the upload queue or the computation queue. Then the task embedding e is denoted by $e = [e_1, e_2, \ldots, e_j, \ldots, e_N]$. Each e_j represents the feature vector of subtask $t_{v,p}$, which includes the offloading index, the size of subtask itself and the required computation resource, the bandwidths of subchannels and the processing rates of processors, denoted as

$$e(t_{v,p}) = [I_{v,p}, d^L_{v,p}, c_{v,p}, f_v, f_1, \ldots, f_M, w_1, \ldots, w_R]. \tag{17}$$

It is noted that the structure of the DAG is dynamic and the length of task embedding e varies with subtask number. In order to learn the implicit relation between different subtasks and their offloading decisions, a dedicated Seq2Seq model is designed to characterize the task embedding e with varying length, which will be described in detail later.

- Action Space: one-hot encoding is adopted to represent the offloading decision at t step, which is represented as an $RM + 1$-dimensional binary vector

$$a_t = [a_{t1}, a_{t2}, \ldots, a_{tk}, \ldots] \tag{18}$$

where each a_{tk}, $k = (r-1) * M + m$, indicates whether the r_{th} subchannel and m_{th} processor is selected or not. Particularly, $a_{t,0} = 1$ indicates the subtask is offloaded to the local vehicle. According to (18), the dimension of action space is 2^{M*R+1}, which scales exponentially with the number of channels and processors. Therefore, the traditional reinforcement learning algorithm, such as Q-learning, cannot efficiently solve the MTO problem due to large action spaces.

- Reward Function: it is computed by the time of completing previous subtask $t_{v',p'}$ minus the time of completing the current offloaded subtask $t_{v,p}$, which is formulated as follows

$$R(t_{v,p}) = \begin{cases} CE(t_{v',p'}) - CE(t_{v,p}),\ if\ I_{v,p} = I_{v',p'} + 1, \\ -CE(t_{v,p}),\ if\ I_{v,p} = 0 \end{cases} \tag{19}$$

Where $I_{v,p}$ denotes the offloading index of $t_{v,p}$ and $t_{v,p}$ is scheduled next to $t_{v',p'}$. Based on (19), the higher reward indicates lower increment of time consumption by completing one more subtask, which contributes to minimizing the objective.

Seq2Seq model is an effective paradigm to encode the sequence of input information into a vector recurrently and decode it to generate the output. However, when the input sequence length is too long, it will cause seriously information loss. To overcome this issue, attention mechanism [38] is added to allow the decoder to show different preferences to different parts of input sequence and alleviates the information loss. As shown in Fig. 3, the proposed Seq2Seq model consists of two parts: encoder and decoder.

- The encoder is a bidirectional gated recurrent units (Bi-GRU), where the forward and backward functions of Bi-GRU are defined as h^f_{enc} and h^b_{enc}. Accordingly, the subtasks are encoded according to the ascending order of the offloading indexes in an recurrent way, obtained by

$$e^f_i = h^f_{enc}(e^f_{i-1}, t_i)$$
$$e^b_i = h^b_{enc}(e^b_{i+1}, t_{i+1})$$
$$e_i = (e^f_i, e^b_i) \tag{20}$$

- The decoder is an attentional GRU, where the function of the GRU is defined as h_{dec}. For the subtask with offloading index j, its output generated by the decoder, denoted by d_j, is jointly determined by the previous offloading action a_{j-1}, the contex vector c_j and previous output d_{j-1}, formulated as follow

$$d_j = h_{dec}(d_{j-1}, c_j, a_{j-1}). \tag{21}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

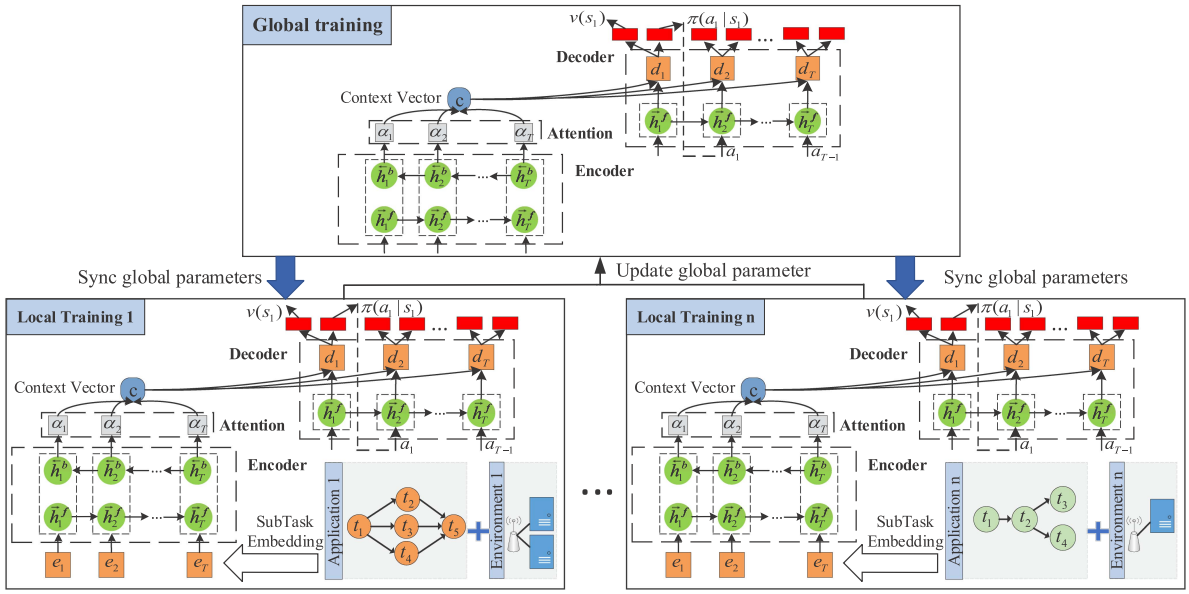


Fig. 3. The Paradigm of SMRL-MTO.

c_j is the context vector based on attention mechanism for predicting the action of subtask with offloading index j and computed by

$$c_j = \sum_{k=1}^{N} \alpha_{jk} e_i. \tag{22}$$

α_{jk} is the weight to measure the connection between the input e_i and the output a_j, which exactly evaluates the inheritance between different subtasks. The attention mechanism in this paper is defined as a trainable feed forward neural network.

B. MRL-Based Model Training

The designed SMRL- MTO adopts the MAML framework, which assumes that the model is parametrized and updated by gradient-based learning technique and able to adapt to a variety of tasks, such as classification, regression and reinforcement learning. The basic idea of MAML is to pretrain meta parameters on a large number of different sampled tasks and fast adapt to a new task based on meta parameters from a small amount of new data. In this paper, we design the MRL-based modeling training by implementing the MAML framework in vehicular edge environment, where the general training procedure consists of local gradient update and meta training and is described as follows. First, the local gradient update is implemented at each terminal and responsible of computing the gradient based on the trajectories of the tasks sampled from task distribution. The generated local gradient will be used for the parameter update of local model and also uploaded to MEC server for later meta training. Further, the meta training performed at MEC server is used for updating meta parameters based on the aggregation of local gradient uploaded from user terminals. Particularly, due to dynamic vehicular environment, the meta parameter is periodically updated. When the terminal faces a new task or the environment varies a lot, a specific offloading policy is obtained with a small number of gradient steps from meta parameters.

The local gradient update is designed based on a policy gradient method, called the Proximal Policy Optimization (PPO) algorithm [39] for searching the offloading policy of the specific MDP. The basic idea of PPO is to transform the on-policy training process into off-policy by importance sampling, so as to speed up the training. There are two different policies existing in PPO: sampling policy $\pi_{\theta'}$ and target policy π_θ. Before each sampling, $\pi_{\theta'}$ is initialized as π_θ. Then, the PPO uses $\pi_{\theta'}$ to generate experience trajectories, which is used for updating π_θ. In addition, for the purpose of stabilizing the training process, the policy objective is designed by synthesizing the PPO-clip and KL divergence as follows

$$J^P(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=1}^{T} \min(clip_{1-\varepsilon}^{1+\varepsilon}(Pr_t)\hat{A}_t, Pr_t\hat{A}_t) \right] - \beta KL(\theta, \theta'), \tag{23}$$

where the KL divergence $KL(\theta, \theta')$ is used to evaluate the difference between π_θ and $\pi_{\theta'}$ for generating even distribution of sampling action and the clip function $clip_{1-\epsilon}^{1+\epsilon}(Pr_t)$ limits the value of Pr_t in the interval of $[1-\epsilon, 1+\epsilon]$ for avoiding over update of parameters. Pr_t represents the probability ratio between the target policy π_θ and sampling policy $\pi_{\theta'}$ in step t, and $\hat{A}_t$ is the general advantage estimator (GAE) used as the advantage function in step t, which are defined as follows

$$Pr_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta'}(a_t|s_t)}, \tag{24}$$

$$\hat{A}_t = \sum_{l=0}^{T-t} (\mathcal{T}\lambda)^l \left[r_{t+l} + \mathcal{T}V(s_{t+l+1}) - V(s_{t+l}) \right], \tag{25}$$

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

where λ is used to balance the bias and variance of advantage function value, and $V(s_t)$ is the value function which indicates the expected return of the experience trajectory starting from s_t. Then, the loss of the value function is defined as follows.

$$J^V(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=1}^{T} (V'(s_t) - V(s_t))^2 \right] \tag{26}$$

Where $V'(s_t) = \sum_{l=0}^{T-t} \mathcal{T}^l r_{t+l}$. Based on (23) and (26), the objective function and the update function of the PPO are defined as follows.

$$J(\theta) = J^P(\theta) - \eta J^V(\theta), \tag{27}$$

$$\hat{\theta} = \theta + \alpha \nabla J(\theta), \tag{28}$$

where η is the coefficient of the loss function of state value and α is the learning rate.

For meta training, we use the MAML framework proposed by [33] to train a model that is fast adaptive to a new task $\mathcal{T}_i$ drawn from a task distribution $\rho(\mathcal{T})$ after a small number of training epochs. The objective of meta-learning is defined as

$$J^M(\theta) = E_{\mathcal{T}_i \sim \rho(\mathcal{T})}[J_{\mathcal{T}_i}(\hat{\theta}_i)],\ \hat{\theta}_i = \theta + \alpha \nabla J_{\mathcal{T}_i}(\theta), \tag{29}$$

where $J_{\mathcal{T}_i}$ is the objective function of task $\mathcal{T}_i$ defined in (27). Then the parameters related to the global model are updated as follows using the gradient ascent method.

$$\theta = \theta + \beta \nabla_\theta J^M(\theta) = \theta + \beta \nabla_\theta E_{\mathcal{T}_i \sim \rho(\mathcal{T})}[J_{\mathcal{T}_i}(\hat{\theta}_i)], \tag{30}$$

where β is the learning rate of meta training. However, the parameter update in (30) involves the computation of the second-order gradient, which brings expensive computation cost and is inefficient. Therefore, we use the first-order approximation to avoid the computation of second-order gradient, which is defined as follows.

$$\nabla_\theta J^M(\theta) \approx g^M = \frac{1}{n} \sum_{i=1}^{n} [(\hat{\theta}_i - \theta)/\alpha/m], \tag{31}$$

where n denotes the number of sampled tasks in meta training and m is the update step for local gradient update.

Overall, the implementation of SMRL-MTO is shown in Algorithm V.1 is described as follows. First, the training data is generated based on the distribution $\rho(\mathcal{T})$, and each set of parameters generates a dataset of DAGs. Then, in the loop of local gradient update, the offloading policy for a specific MTO scenario $\mathcal{T}_i$ is trained. For each $\mathcal{T}_i$, the parameters of sampling and target policy are initialized to the meta policy and experience pool is set to empty, shown in lines 5∼7. For each task in $\mathcal{T}_i$, the SMRL-MTO performs action according to sampling policy, obtains reward and collects the trajectory, shown in lines 10∼12. Then, the parameters of target policy network is updated with m steps based on (27), shown in lines 15∼16. Finally, in the meta training loop, the policy network parameter is updated based on (31), as shown in line 18. When the algorithm terminates, we can acquire the meta policy with pre-trained parameter θ, which adapts to new $\mathcal{T}_i$ with local gradient update with a few steps.

Algorithm V.1. Seq2Seq-Based Meta Reinforcement Learning for Multi-Task Offloading (SMRL-MTO).

Input and initialization:
Initialize n training data sets based on task distribution $\rho(\mathcal{T})$;
Initialize the policy network parameter θ of meta training;

1: Initialize n training data sets based on task distribution $\rho(\mathcal{T})$
2: Initialize the policy network parameter θ of meta training
3: **for** $k = 1$ to K **do**
// **Meta training loop**
4: **for** each MTO scenario $\mathcal{T}_i \sim \rho(\mathcal{T})$ **do**
// **Local gradient update loop**
5: Initialize the network parameter of sampling policy $\theta_i' \leftarrow \theta$
6: Initialize the network parameter of target policy $\theta_i \leftarrow \theta$
7: Initialize the experience pool $\mathcal{D}_i \leftarrow \emptyset$
8: **for** each DAG in $\mathcal{T}_i$ **do**
9: **for** $t = 1, \cdots$ T **do**
10: Sample action a_t for state s_t based on policy $\pi_{\theta_i'}$
11: Obtain reward r_t defined in (19)
12: Put the trajectory (s_t, a_t, r_t) into experience pool $\mathcal{D}_i$
13: **end for**
14: **end for**
15: Compute $J(\theta)$ of target policy based on (27)
16: Update target policy $\hat{\theta}_i = \theta_i + \alpha \nabla_{\theta_i} J_{\mathcal{T}_i}(\theta_i)$ with m steps
17: **end for**
18: Update the meta parameter $\theta = \theta + \beta g^M$
19: **end for**
20: **return**The meta policy of $\rho(\mathcal{T})$ with parameter θ

VI. Performance Evaluation

A. Default Setting

We implemented the system architecture in Section III. We simulate the MEC-based vehicular environment based on Python programming, where the parameters of both MEC servers and vehicular tasks are sampled from a given distribution and the offloading procedure in system model described in Section III is implemented including task transmission, queuing and computation. The detailed parameter setting is presented as follows. Specifically, the wireless uplink channel of MEC server is divided into four subchannels and the bandwidth of each subchannels is evenly generated from the interval $[3, 6]$ MHz. The channel noise power is set as 10^{-5} mW and channel gain ranges from 1 to 3. Further, each MEC server is equipped with three processors, where the processing rate is evenly generated from the interval $[1, 2]G$ CPU cycles. In addition, for each vehicle, the processing rate and the transmission power are

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

randomly selected from the interval $[0.5, 1]$ G CPU cycle/s and $[100, 200]$ mW, respectively. To characterize different types of vehicular applications, the dataset of vehicular tasks is generated based on a synthetic DAG generator called DAGGEN [40], which is used for generating heterogeneous DAGs. Specifically, four parameters are set to control task topology and profiles, where n, fat, $density$ and ccr determine the subtask number, the height and width of the DAG, the density of edges, and the ratio of output data size to the size of subtask itself, respectively. By default, n, $density$, fat and ccr are set to 5, 0.8, 0.5, and 0.5, respectively. The size of subtask itself and the required computing resources are randomly selected from the intervals $[300, 400]$ KB and $[50, 60]$ M CPU cycles. For model pretraining, training dataset is sampled based on a set of predefined parameters. Then, the model is evaluated on testing dataset generated by different parameter settings, which represents for new tasks or environment settings.

For algorithm implementation, there are three layers in both encoder and decoder and each layer has 512 hidden units. The discount factors $\mathcal{T}$ and λ are set as 0.95 and the clip constant is set as 0.2. The weights of KL divergence and state value loss are set as 0.1 and 0.5, respectively. In addition, the learning rate is both set to 10^{-4}. The SMRL-MTO algorithm is first trained offline to learn a meta policy based on training data set generated by the synthetic DAG generator. Further, the SMRL-MTO with meta policy performs on new task offloading scenarios with unseen parameters.

For performance comparison, we implement the following competitive algorithms:

- PPO [41]: which is implemented at each MEC server and makes offloading decision for subtasks. In this way, each agent has an independent neural network.
- DQN [42]: which is also implemented at each MEC server. Each DQN model train it's network parameter using it's local experience memory.
- Greedy: which greedily chooses the processor or uploads subchannel which has lowest load for pending subtasks.

For performance evaluation, we collected the following statistics for each subtask $t_{v,p}$: the execution time $ET(t_v)$, the waiting time $CW(t_{v,p})$ in the computation queue and the offloading decision $x^0_{v,p}$, the computation load cl_m of processor m, the computation load cl_v of vehicle v and the transmission load tl_r of subchannel r. Besides the objective, we define extra four metrics as follows.

- Computation waiting time (CWT): it is defined as the sum of all subtask's waiting time in computation queue divided by the subtask number, i.e.,

$$CWT = \sum_{v\in V}\sum_{p\in P}\frac{CW(t_{v,p})}{||V||\times||P||} \tag{32}$$

- Offloading proportion (OP): let $OP(l)$ and $OP(m)$ denote the ratio of subtasks offloaded to local vehicle and MEC server, respectively, i.e.,

$$OP(l) = \sum_{v\in V}\sum_{p\in P}\frac{x^0_{v,p}}{||V||\times||P||}$$

$$OP(m) = 1 - OP(l) \tag{33}$$

- Load balance ratio (LBR): the Jain's fairness index [43] is used to evaluate the load balance of processors and subchannels, denoted by $LBR(c)$ and $LBR(r)$, i.e.,

$$LBR(c) = \frac{\left(\sum_{v\in V}\frac{cl_v}{f_v}+\sum_{m\in M}\frac{cl_m}{f_m}\right)^2}{(||V||+||M||)\left(\sum_{v\in V}\left(\frac{cl_v}{f_v}\right)^2+\sum_{m\in M}\left(\frac{cl_m}{f_m}\right)^2\right)} \tag{34}$$

$$LBR(r) = \frac{\left(\sum_{r\in R} tl_r/w_r\right)^2}{||R||\sum_{r=1}^{R}\left(tl_r/w_r\right)^2} \tag{35}$$

  Higher value of LBR indicates that the workload is more evenly distributed.
- Fairness of Vehicles (FoV): to evaluate the fairness among multiple vehicles, Fairness of Vehicles (FoV) is defined as the summation of the square of execution time $ET(t_v)$ divided by the square of the summation of execution time $ET(t_v)$based on the Jain's fairness index [43], which is formulated as

$$FoV = \frac{\sum\limits_{v\in V}\left(ET(t_v)\right)^2}{\left(\sum\limits_{v\in V}ET(t_v)\right)^2} \tag{36}$$

  The higher value of FoV indicates that the fluctuation of the execution time among vehicles is smoother and brings better fairness. Particularly, $FoV = 1$ indicates the optimal fairness where the execution time of each vehicle is equal to each other.

B. Simulation Results

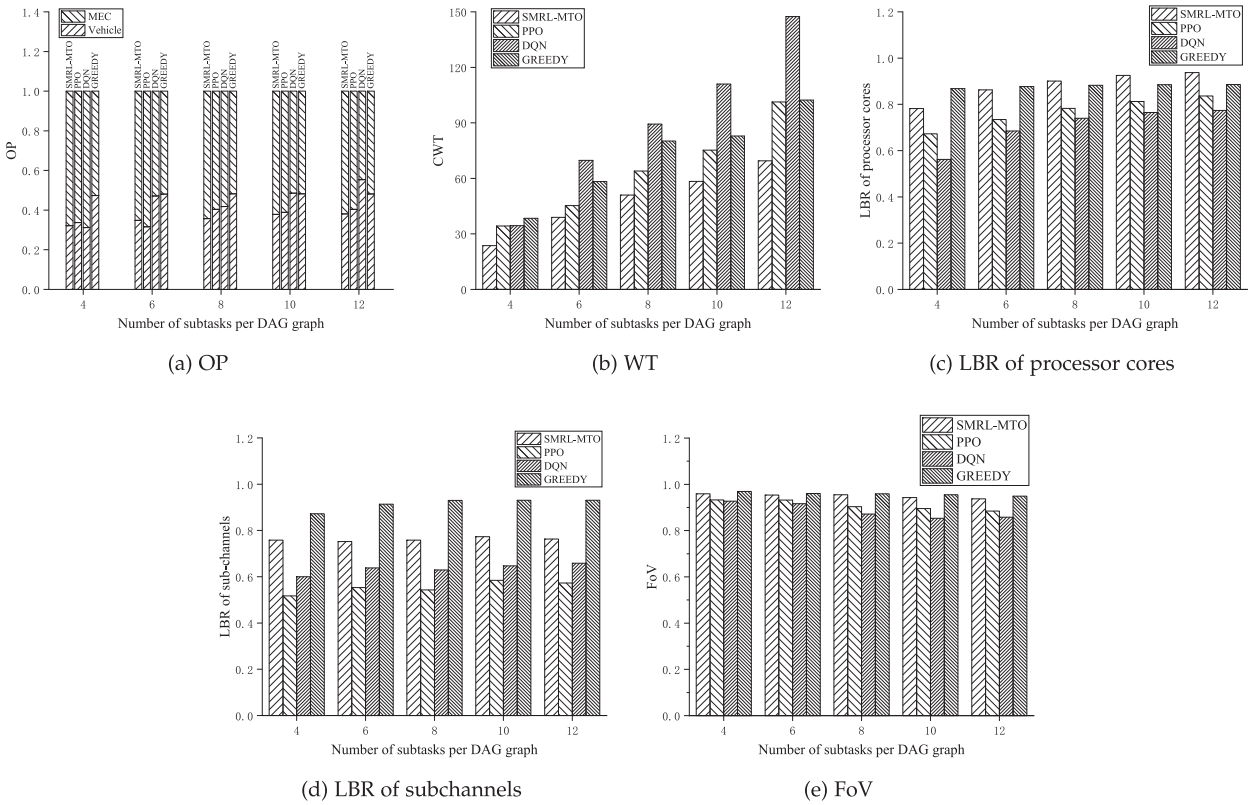
1) Effect of Subtask Number: In this section, we evaluate the performance of four algorithms under different subtask numbers, where five testing sets are generated with subtask number $n \in [4, 6, 8, 10, 12]$ and each set consists of 50 DAGs. Particularly, the SMRL-MTO, PPO and DQN are first pre-trained on the training datasets with $n \in [5, 7, 9]$ with 1000 gradient update steps, where the obtained meta-policy is used as meta parameter performed on testing set. Table II compares the AET of Greedy, SMRL-MTO, PPO and DQN with 20 and 50 training steps under different subtask numbers, respectively. According to Table II, the AET of four algorithms increases when the subtask number increases. It is because that higher subtask number brings higher workload of each task and results in longer waiting time before execution, which is validated by Fig. 4(b) that the CWT of four algorithms increases.

Further, the proposed SMRL-MTO achieves the lowest AET among four algorithms according to Table II, which is explained as follows. First, the SMRL-MTO can fully exploit the communication and computation resources and achieve load balance between multiple subchannels and processors. As shown in Fig. 4(c) and (d), the SMRL-MTO achieves competitive LBR compared with Greedy and higher than that of PPO and DQN. Second, the SMRL-MTO can better learn and exploit the dependency between subtasks based on dedicated designed Seq2seq

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE AET OF FOUR ALGORITHMS UNDER DIFFERENT SUBTASK NUMBERS

Subtask number	Greedy	SMRL-MTO		PPO		DQN	
		update steps(5)	update steps(20)	update steps(5)	update steps(20)	update steps(5)	update steps(20)
4	274.94	242.53	236.48	258.66	259.27	260.60	257.68
6	359.96	327.46	322.57	360.41	357.38	395.20	375.97
8	454.81	410.25	403.33	453.32	452.57	511.33	491.56
10	474.23	430.25	420.33	486.33	487.30	602.28	581.91
12	567.71	513.96	513.41	585.17	584.09	713.87	673.68



(a) OP (b) WT (c) LBR of processor cores

(d) LBR of subchannels (e) FoV

Fig. 4. The performance of four algorithms under different subtask numbers.

model and effectively reduce waiting time before execution, which is reflected by the lowest CWT achieved by SMRL-MTO compared with other algorithms shown in Fig. 4(b). Though the Greedy achieves the highest LBR, but it still suffers from high CWT since it cannot learn the relationship between subtasks. Third, the SMRL-MTO can fully utilize the resources of MEC servers for accelerating the task execution. As shown in Fig. 4(a), the SMRL-MTO achieves the highest $OP(m)$ and maintains at the stable level when the subtask number increases, which indicates the SMRL-MTO can well balance the workload distribution among the MEC server and local vehicle.

To show fast adaptiveness, we compare SMRL-MTO, PPO and DQN with two different gradient update steps. As shown in Table II, the SMRL-MTO achieves competitive AET with only 5 update steps compared to the result with 20 update steps and outperforms PPO and DQN with 20 update steps, which indicates that the pre-trained meta-policy enables SMRL-MTO fast adapt to new MTO scenario. Particularly, we compare the convergence performance of four algorithms when adapting to new task environments with different subtask number. It is observed in Fig. 5(a) that the AET of the four algorithms is very close to each other at first. Then, the SMRL-MTO decreases the AET dramatically within a few gradient update steps and converge to the best policy among the four algorithms. The Greedy maintains the same AET since its policy is fixed and cannot learn from the environment. The PPO has a faster convergence speed than the DQN. In all cases, i.e., Fig. 5(a)∼(e), it is observed that the DQN cannot fast adapt to a new task environment, where the DQN achieves the worst AET at first and requires the longest training steps for convergence compared with PPO and SMRL-MTO. The PPO can converge very fast but cannot search the best policy compared with the SMRL-MTO since it cannot well exploit the relationship between different subtasks. It is noted that the SMRL-MTO can obtain the best policy within 20 gradient update steps compared with the other

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

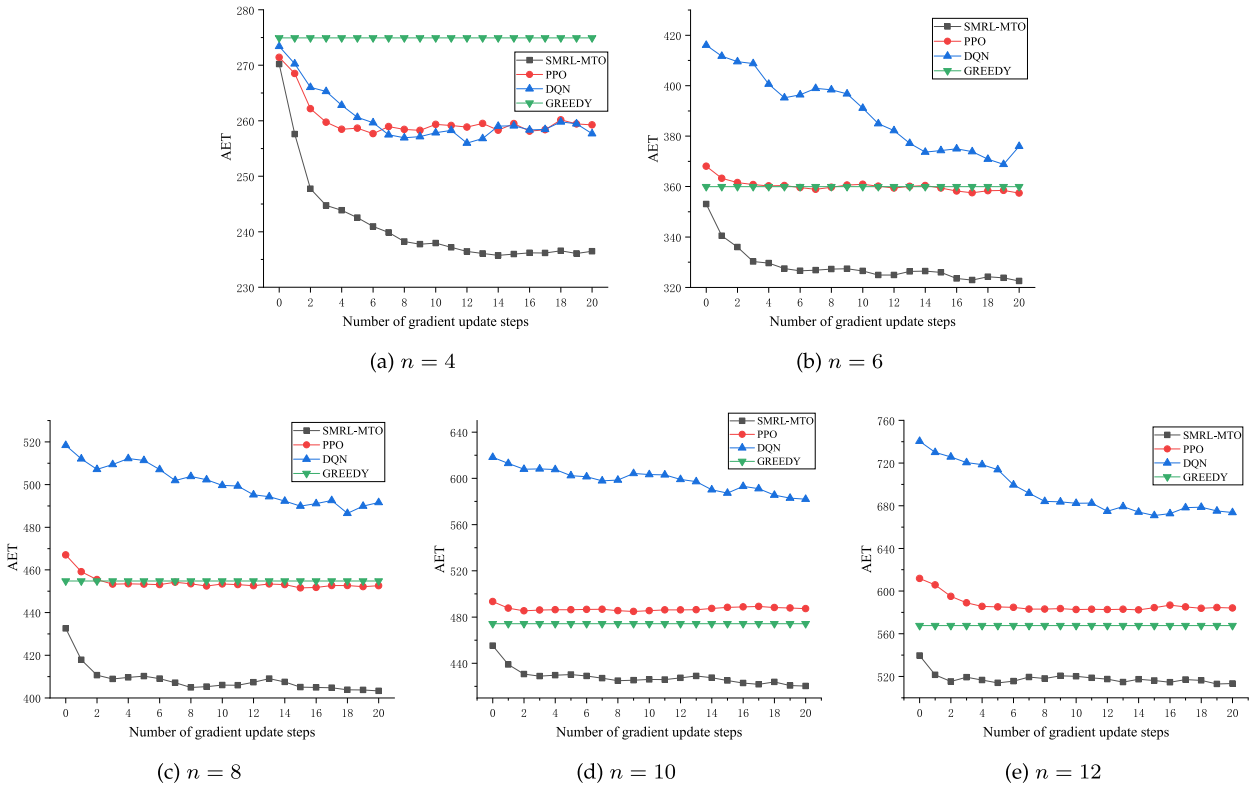


Fig. 5. The convergence performance of four algorithms under different subtask numbers.

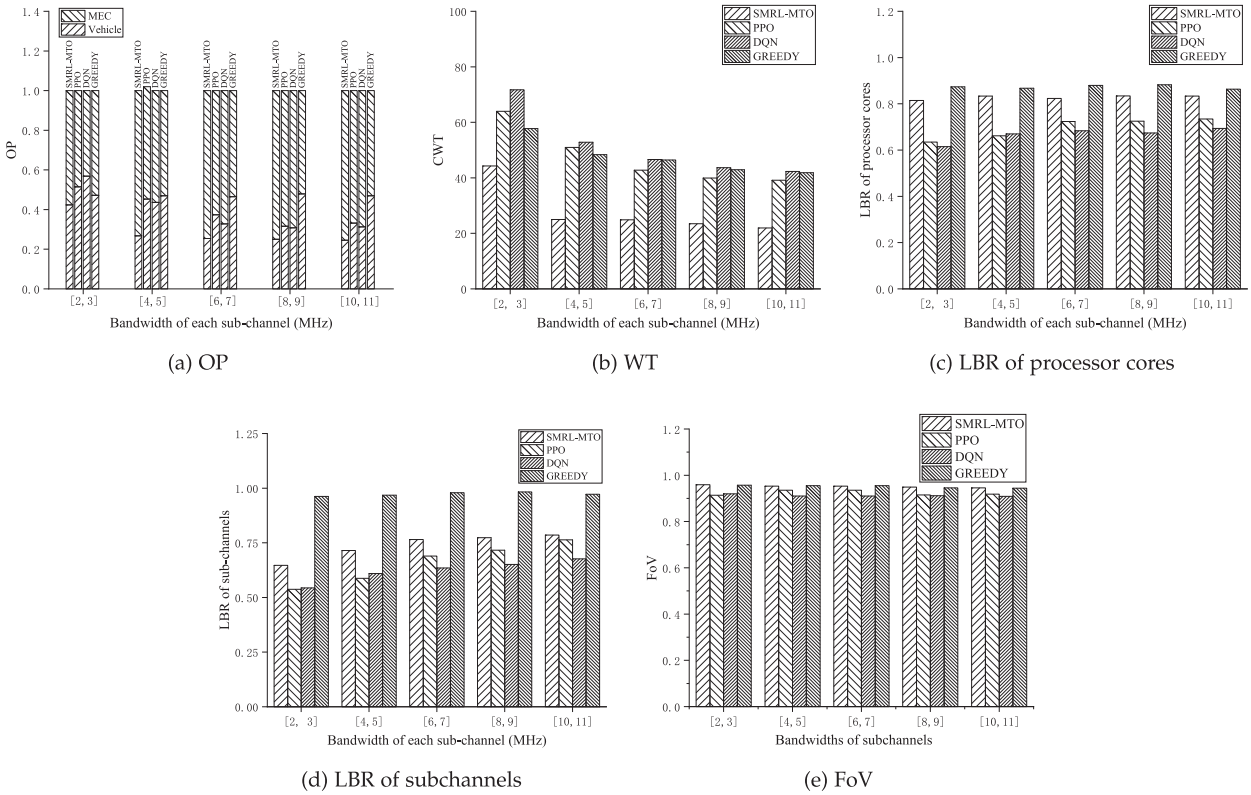
TABLE III
THE AVERAGE EXECUTION TIME (AET) OF FOUR ALGORITHMS ON DIFFERENT BANDWIDTHS OF SUBCHANNEL

Bandwidth of subchannel (MHz)	Greedy	SMRL-MTO		PPO		DQN	
		update steps(5)	update steps(20)	update steps(5)	update steps(20)	update steps(5)	update steps(20)
[2, 3]	374.15	339.77	336.68	370.93	368.87	398.89	383.35
[4, 5]	315.74	274.34	272.39	321.67	322.40	373.56	356.74
[6, 7]	294.23	252.38	250.87	297.10	296.38	342.94	339.87
[8, 9]	287.58	246.89	240.37	279.78	280.39	340.73	327.39
[10, 11]	278.02	231.73	228.24	274.23	273.94	328.97	310.67

algorithms. In addition, we compare the fairness among multiple vehicles in terms of AET. Fig. 4(e) shows the FoV of the four algorithms under different subtask numbers. It is noted that the SMRL-MTO achieves the second highest FoV close to 1.0 among the four algorithms, which indicates that the AET does not fluctuate among vehicles and avoids the overlong execution time of individual vehicles. Though the Greedy achieves the best FoV but with large performance gap of AET compared with the SMRL-MTO.

2) Effect of Wireless Bandwidth: In this section, we compare the performance of four algorithms under different bandwidths of subchannels. Specifically, the SMRL-MTO, PPO and DQN are pretrained on the datasets with bandwidth $w_r \in [3, 4], [5, 6], [7, 8]$ within 1000 gradient update steps. Then, all algorithms are tested on five datasets with $w_r \in [2, 3], [4, 5], [6, 7], [8, 9], [10, 11]$. Table III compares the AET of four algorithms with different update steps under varying bandwidths of subchannel. The increasing subchannel bandwidth brings lower data transmission time, which reduces communication overhead between local vehicles and MEC servers. Accordingly, the AET of four algorithms decreases as the bandwidth of subchannel increases. Particularly, the decreasing speed of the AET slows down when the bandwidth of subchannel keeps increasing. It is because that the transmission time dominates the AET at first with limited wireless bandwidth. When the bandwidth keeps increasing, the computation time becomes the dominant and the effect of transmission time reduction becomes insignificant, which is validated by Fig. 6(a) and (b) that the $OP(l)$ and the CWT of four algorithms decreases dramatically from $w_r \in [2, 3]$ to [4,5] and then maintains at a stable level. It is noted that the SMRL-MTO still achieves the lowest CWT among four algorithms. As shown in Fig. 6(c) and (d), the SMRL-MTO can achieve the competitive LBR of subchannels and processors compared with Greedy, which indicates that SMRL-MTO also achieves well workload balance. Further, with respect to fairness, as shown in Fig. 6(e), the SMRL-MTO keeps the FoV

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.



(a) OP (b) WT (c) LBR of processor cores

(d) LBR of subchannels (e) FoV

Fig. 6. The performance of four algorithms under different bandwidths of subchannels.

at the stable level (around 0.95) and competitive to the Greedy under varying bandwidth of subchannel, which shows that the SMRL-MTO can fairly balance the resource allocation among multiple vehicles in dynamic network conditions. Accordingly, the SMRL-MTO achieves the best AET among four algorithms. In addition, the SMRL-MTO with 5 training steps achieves competitive AET compared that with 20 training steps and even lower than DQN and PPO with 20 training steps, which indicates that SMRL-MTO can fast adapt to varying network conditions.

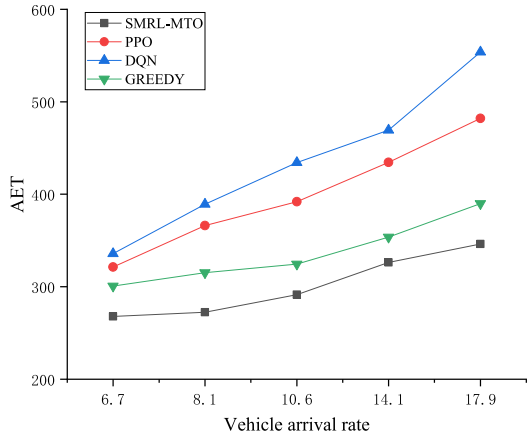
3) Effect of Vehicle Arrival Rate: This experiment compares the performance of four algorithms under different vehicular arrival rates, where the SMRL-MTO is trained on the datasets with vehicular arrival rate belongs to the set {3, 9, 15, 21, 27, 33} and all algorithms are tested using the realistic vehicular trajectories. To test algorithm performance in different traffic scenarios, the realistic vehicular trajectories [44] are extracted from the Qingyang District, Chengdu, 8:00 to 16:00 on 11th, October, 2018. According to the statistics, we select five representative traffic periods, which generates five testing datasets characterized by different arrival rates, i.e., {6.7, 8.1, 10.6, 14.1, 17.9}. Each algorithm is scheduled for offloading the pending tasks in every 20 seconds. Fig. 7(a) compares the AET of all algorithms under different vehicle arrival rates. As shown, the AET of all the algorithms increases when vehicle arrival rate increases, which indicates heavier workload. Fig. 7(b) shows the FoV of four algorithms under varying vehicle arrival rates. It is observed that the FoV of PPO and DQN vary dynamically with different traffic workloads, which indicates that the PPO and DQN cannot fairly handle multiple vehicular tasks in dynamic traffic workloads. The SMRL-MTO keeps the FoV at a preferable level, since it can treat each vehicular task equally by encoding all these subtasks based on the designed Seq2seq model. Accordingly, the proposed SMRL-MTO still achieves the best performance compared to the other three algorithms, which demonstrates the scalability of our proposed algorithm against varying traffic workloads of real-world vehicular trajectories.

TABLE IV
RUNNING TIME OF FOUR ALGORITHMS

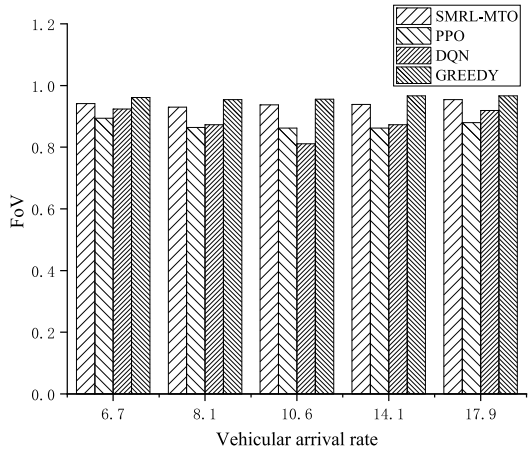
	SMRL-MTO	PPO	DQN	Greedy
Pre-Training	47.54min	13.98min	19.92min	/
Update step(5)	0.25min	0.13min	0.14min	/
Update step(20)	0.78min	0.44min	0.56min	/
Decision	0.029s	0.0296s	0.0288s	0.0288s

4) Discussion of Running Time: In this section, we discuss running time of four algorithms, which is one of the most important metrics to evaluate algorithm feasibility from engineering aspect. The simulation platform is a computation server with Intel Xeon Gold 6248R Dual CPU, 128 GB RAM and NVIDIA GeForce RTX 3090. Table IV shows the running time of four algorithms during the phase of pre-training, fast adaptation and decision making. In pre-training phase, the SMRL-MTO, PPO and DQN are trained with 1000 gradient update steps, which

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.



(a) AET



(b) FoV

Fig. 7. The performance of four algorithms under different vehicle arrival rates.

take the running time of 47.54min, 13.98min and 19.92min. It can be seen that SMRL-MTO takes the most time consumption due to deep structure of designed neural networks. Fortunately, it is noted that the pre-training of the proposed algorithm can be performed offline with sampled data from historical observation or built distribution. In fast adaptation phase, as shown in Table IV, the SMRL-MTO achieves competitive running time compared with PPO and DQN. Particularly, the running time of SMRL-MTO is only a slightly higher than PPO and DQN but falls within one minute in both 5 and 20 gradient steps. It is also noted in Fig. 5, the SMRL-MTO can always fast adapt to new task offloading scenario within 5 gradient steps at the cost of 0.25min running time. Moreover, the running time of decision making after model training is averaged over 1000 tasks. As shown in Table IV, the running time of decision making of all these four algorithms are almost the same and around 0.029s, which indicates that the proposed algorithm can run as fast as heuristic algorithm and validates its applicability in realistic vehicular applications.

VII. Conclusion

This paper investigated the MTO problem, where each scenario $\mathcal{T}_i$ is characterized by the features of both vehicular tasks V and MEC servers M, such as task topologies, data dependencies between subtasks, resource requirements and transmission/computation capabilities of MEC servers. To fast adapt to new MTO scenario, we developed the SMRL-MTO algorithm by integrating Seq2seq model and meta-learning framework. Specifically, the Seq2seq model developed a Bi-GRU to explore the dependency relationship between subtasks by encoding sequential actions iteratively and make offloading action by evaluating the weight of different part of input based on attention mechanism. Further, a meta reinforcement learning framework is proposed to train a meta policy based on PPO in meta training process and fast adapt to a new task scenario within a few gradient steps. Finally, we compared the SMRL-MTO with three competitive algorithms, where the results demonstrate the fast adaptiveness and the effectiveness of the proposed algorithm in varying service scenarios.

It is noted that the proposed algorithm still requires large sampled data for meta training. There exist many emerging meta-RLs that can improve sample efficiency and generate better meta-policy by introducing latent space model, such as MAESN [35] and PEARL [36]. In the future, we will investigate the feasibility of incorporating theses mechanisms into our algorithm to further reduce sample overhead and achieve better adaptation. In addition, we will test the performance of other training methods, such as TD3 and DPPO, for further enhancing training performance.

References

[1] T. Xiao, C. Chen, Q. Pei, and H. H. Song, "Consortium blockchain-based computation offloading using mobile edge platoon cloud in Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17 769–17 783, Oct. 2022.

[2] M. Abbasi, M. Yaghoobikia, M. Rafiee, M. R. Khosravi, and V. G. Menon, "Optimal distribution of workloads in cloud-fog architecture in intelligent vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4706–4715, Jul. 2021.

[3] W. Fan et al., "Joint task offloading and resource allocation for vehicular edge computing based on V2I and V2V modes," *IEEE Trans. Intell. Transp. Syst.*, early access, Jan. 05, 2023, doi: 10.1109/TITS.2022.3230430.

[4] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.

[5] P. Dai, F. Song, K. Liu, Y. Dai, P. Zhou, and S. Guo, "Edge intelligence for adaptive multimedia streaming in heterogeneous Internet of Vehicles," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1464–1478, Mar. 2023.

[6] K. Liu, K. Xiao, P. Dai, V. C. Lee, S. Guo, and J. Cao, "Fog computing empowered data dissemination in software defined heterogeneous VANETs," *IEEE Trans. Mobile Comput.*, vol. 20, no. 11, pp. 3181–3193, Nov. 2021.

[7] C. Zhang et al., "Online approximation scheme for scheduling heterogeneous utility jobs in edge computing," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 352–365, Feb. 2023.

[8] P. Dai, K. Hu, X. Wu, H. Xing, and Z. Yu, "Asynchronous deep reinforcement learning for data-driven task offloading in MEC-empowered vehicular networks," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.

[9] X. Kong et al., "Deep reinforcement learning-based energy-efficient edge computing for Internet of Vehicles," *IEEE Trans. Ind. Inform.*, vol. 18, no. 9, pp. 6308–6316, Sep. 2022.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.

[10] H. Shahzad and T. H. Szymanski, "A dynamic programming offloading algorithm using biased randomization," in *Proc. IEEE 9th Int. Conf. Cloud Comput.*, 2016, pp. 960–965.
[11] A. Ashok, P. Steenkiste, and F. Bai, "Vehicular cloud computing through dynamic computation offloading," *Comput. Commun.*, vol. 120, pp. 125–137, 2018.
[12] Z. Xiao et al., "Vehicular task offloading via heat-aware MEC cooperation using game-theoretic method," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 2038–2052, Mar. 2020.
[13] M. Huang, W. Liu, T. Wang, A. Liu, and S. Zhang, "A cloud–MEC collaborative task offloading scheme with service orchestration," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5792–5805, Jul. 2020.
[14] Y. Wang, X. Hu, L. Guo, and Z. Yao, "Research on V2I/V2V hybrid multi-hop edge computing offloading algorithm in IoV environment," in *Proc. IEEE 5th Int. Conf. Intell. Transp. Eng.*, 2020, pp. 336–340.
[15] V. H. Hoang, T. M. Ho, and L. B. Le, "Mobility-aware computation offloading in MEC-based vehicular wireless networks," *IEEE Commun. Lett.*, vol. 24, no. 2, pp. 466–469, Feb. 2020.
[16] J. Long, Y. Luo, X. Zhu, E. Luo, and M. Huang, "Computation offloading through mobile vehicles in IoT-edge-cloud network," *EURASIP J. Wirel. Commun. Netw.*, vol. 2020, no. 1, pp. 1–21, 2020.
[17] Z. Kuang, Z. Ma, Z. Li, and X. Deng, "Cooperative computation offloading and resource allocation for delay minimization in mobile edge computing," *J. Syst. Architecture*, vol. 118, 2021, Art. no. 102167.
[18] Z. Ning et al., "Deep reinforcement learning for intelligent Internet of Vehicles: An energy-efficient computational offloading scheme," *IEEE Trans. Cogn. Commun. Netw.*, vol. 5, no. 4, pp. 1060–1072, Dec. 2019.
[19] C. Guo, L. Liang, and G. Y. Li, "Resource allocation for high-reliability low-latency vehicular communications with packet retransmission," *IEEE Trans. Veh. Technol*, vol. 68, no. 7, pp. 6219–6230, Jul. 2019.
[20] T. Jin, W. Zheng, X. Wen, X. Chen, and L. Wang, "Optimization of computation resource for container-based multi-MEC collaboration system," in *Proc. IEEE 30th Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, 2019, pp. 1–7.
[21] J. Zhou, F. Wu, K. Zhang, Y. Mao, and S. Leng, "Joint optimization of offloading and resource allocation in vehicular networks with mobile edge computing," in *Proc. 10th Int. Conf. Wirel. Commun. Signal Process.*, 2018, pp. 1–6.
[22] J. Zhang, W. Xia, F. Yan, and L. Shen, "Joint computation offloading and resource allocation optimization in heterogeneous networks with mobile edge computing," *IEEE Access*, vol. 6, pp. 19 324–19 337, 2018.
[23] A. A. Al-Habob, A. Ibrahim, O. A. Dobre, and A. G. Armada, "Collision-free sequential task offloading for mobile edge computing," *IEEE Commun. Lett.*, vol. 24, no. 1, pp. 71–75, Jan. 2020.
[24] Y. Han, Z. Zhao, J. Mo, C. Shu, and G. Min, "Efficient task offloading with dependency guarantees in ultra-dense edge networks," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
[25] X. Xu, X. Zhang, X. Liu, J. Jiang, L. Qi, and M. Z. A. Bhuiyan, "Adaptive computation offloading with edge for 5G-envisioned Internet of Connected Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5213–5222, Aug. 2021.
[26] P. Dai, K. Hu, X. Wu, H. Xing, F. Teng, and Z. Yu, "A probabilistic approach for cooperative computation offloading in MEC-assisted vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 899–911, Feb. 2022.
[27] Z. Ning et al., "When deep reinforcement learning meets 5G-enabled vehicular networks: A distributed offloading framework for traffic big data," *IEEE Trans. Ind. Informat.*, vol. 16, no. 2, pp. 1352–1361, Feb. 2020.
[28] R. Zhou et al., "Online task offloading for 5G small cell networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 2103–2115, Jun. 2022.
[29] Q. Qi et al., "Knowledge-driven service offloading decision for vehicular edge computing: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol*, vol. 68, no. 5, pp. 4192–4203, May 2019.
[30] X. Wang, Z. Ning, S. Guo, and L. Wang, "Imitation learning enabled task scheduling for online vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 2, pp. 598–611, Feb. 2022.
[31] Z. Yan, P. Cheng, Z. Chen, B. Vucetic, and Y. Li, "Two-dimensional task offloading for mobile networks: An imitation learning framework," *IEEE/ACM Trans. Netw.*, vol. 29, no. 6, pp. 2494–2507, Dec. 2021.
[32] Z. Xu, H. van Hasselt, and D. Silver, "Meta-gradient reinforcement learning," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA: Curran Associates Inc., 2018, pp. 2402–2413.
[33] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.
[34] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, "RL^2: Fast reinforcement learning via slow reinforcement learning," 2017. [Online]. Available: https://openreview.net/forum?id=HkLXCE9lx
[35] A. Gupta, R. Mendonca, Y. Liu, P. Abbeel, and S. Levine, "Meta-reinforcement learning of structured exploration strategies," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 5307–5316.
[36] K. Rakelly, A. Zhou, C. Finn, S. Levine, and D. Quillen, "Efficient off-policy meta-reinforcement learning via probabilistic context variables," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 5331–5340.
[37] J. Wang, J. Hu, G. Min, A. Y. Zomaya, and N. Georgalas, "Fast adaptive task offloading in edge computing based on meta reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 242–253, Jan. 2021.
[38] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.
[39] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv: 1707.06347*. [Online]. Available: http://arxiv.org/abs/1707.06347
[40] F. Suter, "DAGGEN: A synthetic task graph generator," 2013. [Online]. Available: https://github.com/frs69wq/daggen
[41] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[42] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
[43] R. K. Jain et al., "A quantitative measure of fairness and discrimination," Eastern Res. Lab., Digit. Equip. Corporation, Hudson, MA, USA, vol. 21, 1984.
[44] Data source: Vehicle trajectory, 2023. [Online]. Available: https://drive.google.com/drive/folders/1Jot2LF1p8bvfgMJYUxccppfD6fgO6rfc



Penglin Dai (Member, IEEE) received the BS degree in mathematics and applied mathematics and the PhD degree in computer science from Chongqing University, Chongqing, China, in 2012 and 2017, respectively. He is currently an assistant professor with the School of Information Science and Technology, and the associate director of Research Center for Intelligent Sensing, Southwest Jiaotong University, Chengdu, China. His research interests include edge computing, vehicular cyber-physical systems, and intelligent transportation systems.

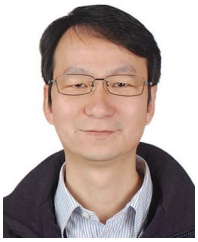


Yaorong Huang received the BE degree in economics from Southwest Jiaotong University, Chengdu, China, in 2020, where he is currently working toward the master's degree with the School of Computing and Artificial Intelligence, Southwest Jiaotong University. His research interests include vehicular networks and blockchain.



Kaiwen Hu received the BE degree in vehicle engineering from Southwest Jiaotong University, Chengdu, China, in 2019. He is currently a research assistant with the School of Information Science and Technology, Southwest Jiaotong University. His research interests include vehicular networks and mobile edge computing.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.



Xiao Wu (Member, IEEE) received the BEng and MS degrees in computer science from Yunnan University, Yunnan, China, in 1999 and 2002, respectively, and the PhD degree in computer science from the City University of Hong Kong, Kowloon Tong, Hong Kong, in 2008. He is currently a professor and an assistant dean with the School of Information Science and Technology, Southwest Jiaotong University, Chengdu, China. He was at the Institute of Software, Chinese Academy of Sciences, Beijing, China, from 2001 to 2002. He was a research assistant and a senior research associate at the City University of Hong Kong, Hong Kong, from 2003 to 2004, and 2007 to 2009, respectively. He was at the School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, USA, and also at the School of Information and Computer Science, University of California, Irvine, CA, USA as a visiting scholar from 2006 to 2007 and 2015 to 2016, respectively. He has authored/co-authored more than 70 research papers in well-respected journals, such as *IEEE Transactions on Image Processing*, *IEEE Transactions on Multimedia*, *IEEE Transactions on Medical Imaging* and prestigious proceedings like Conference on Computer Vision and Pattern Recognition and ACM Multimedia. His research interests include artificial intelligence, multimedia information retrieval, image/video computing, and computer vision. He received the Second Prize of Natural Science Award of the Ministry of Education, China in 2016 and the Second Prize of Science and Technology Progress Award of Henan Province, China, in 2017.



Huanlai Xing (Member, IEEE) received the BEng degree in communications engineering from Southwest Jiaotong University, Chengdu, China, in 2006, the MSc degree in electromagnetic fields and wavelength technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2009, and the PhD degree in computer science from the University of Nottingham, Nottingham, U.K., in 2013. He is an associate professor with the School of Information Science and Technology, Southwest Jiaotong University. His research interests include evolutionary computation, cloud computing, multiobjective optimization, network function virtualization, and software defined networks. He has authored and co-authored more than 30 peer-reviewed journal and conference papers. He is a member of the ACM.



Zhaofei Yu (Member, IEEE) received the BS degree from the Hong Shen Honors School, College of Optoelectronic Engineering, Chongqing University, Chongqing, China, in 2012, and the PhD degree from the Automation Department, Tsinghua University, Beijing, China, in 2017. He is currently an assistant professor with the Institute for Artificial Intelligence, Peking University, Beijing. His current research interests include artificial intelligence, brain-inspired computing, and computational neuroscience.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 13:03:36 UTC from IEEE Xplore. Restrictions apply.